



Operation Black Beacon

Captain Jack Sparrow's Last Haul — Solution Manual

Captain & Lore: Captain Jack Sparrow

Target Architecture: Web Security, Client-Side Reversing, WebAssembly, Cryptography

Difficulty: Medium

Format: Multi-Layered Flag Recovery & Master SHA-256 Reassembly

Environment: Node.js HTTP Server, Service Workers, IndexedDB, WASM

CTF Navigation & Detailed Page URL Map

Operation Black Beacon consists of four distinct web pages/routes, each hosting specific security challenges and mechanisms. Below is the complete URL map with source file mappings:

Method / Route	Target URL	Primary Source Files	Challenge Mechanism
GET /	http://localhost:3000/	index.html gate.js	Layer 0 DevTools Gate & Layer 1 Base91 / CSS delays
GET / archive	http://localhost:3000/pages/archive.html	pages/archive.html js/layer3.js	Layer 2 Role Elevation & IndexedDB Log Store
GET / beacon	http://localhost:3000/pages/beacon.html	pages/beacon.html js/wasm-loader.js wasm/vault.wasm	Layer 3 Canvas Pixel Math & WebAssembly Engine
GET /final	http://localhost:3000/final/index.html	final/index.html	Master Flag Assembly & Final Vault Chamber
POST /api/verify	http://localhost:3000/api/verify	api/server.js	Server Rate-Limited Master Flag Validation API

Learning Note: Notice how the CTF uses a **Service Worker** (`sw.js`). When visiting `/beacon` in a browser, the Service Worker intercepts the request locally and injects synthetic HTTP headers before returning `pages/beacon.html` .



Step-by-Step Solution & Learning Breakdown

Phase 1: Layer 0 — Anti-Inspect Gate Bypass

URL: `http://localhost:3000/` | **File:** `gate.js`

When you land on the page, the chart room is sealed by an anti-inspect overlay (`#lock-overlay`). The script `gate.js` monitors window dimension deltas and blocks standard DevTools key shortcuts (`F12` , `ctrl+Shift+I` , right-click).

Exploitation:

```
// Invoke the exposed override function in the browser console
window.__beacon_override('BLACKBEARD');
```

Result: The sealed overlay dissolves, unlocking access to the main deck and setting `Layer 0` progress.

Phase 2: Layer 1 — Base91 Decoding & Fragment I

URL: `http://localhost:3000/index.html` | **File:** `js/layer1.js`

Inspect the HTML source code of `index.html` to find an embedded comment and inline CSS styles:

```
<!-- LOG FRAGMENT 1 (HTML Comment): *Djd#{]@ -->
<style>
.ember-j { animation-delay: 74ms; } /* ASCII 74 = J */
.ember-a { animation-delay: 65ms; } /* ASCII 65 = A */
.ember-c { animation-delay: 67ms; } /* ASCII 67 = C */
.ember-k { animation-delay: 75ms; } /* ASCII 75 = K */
</style>
```

The ASCII delays spell out **JACK**, giving Captain Jack Sparrow's cipher hint. Decode the comment string using the exposed `_sparrowDecode` helper in console:

```
_sparrowDecode("*Djd#{]@");
```

Fragment I: SPARROW{anchor_in_the_deep}

Phase 3: Layer 2 — IndexedDB Analysis & Fragment II

URL: `http://localhost:3000/pages/archive.html` | **File:** `js/layer3.js`

Navigate to `/pages/archive.html`. To view authentic log records, your client-side rank must be elevated from `guest` to `quartermaster`.

Exploitation:

```
// Set role in LocalStorage and reload
localStorage.setItem('sparrow_ledger_role', 'quartermaster');
location.reload();
```

Once elevated, open DevTools -> **Application** -> **IndexedDB** -> **VaneArchive** -> **logs**. Inspect entry `log-1708` to find the decrypted payload:

Fragment II: SPARROW{ghost_ship_indexed}

Phase 4: Layer 3 — WebAssembly & Canvas Pixel Math (Fragment III)

URL: `http://localhost:3000/pages/beacon.html` | **Files:** `js/canvas-puzzle.js`, `wasm/vault.wasm`

1. Inspect `images/logo.svg` metadata to find the row indicator: `row="0x2a"` (Row 42).
2. Extract the painted RGB byte array on `#flag-canvas` row 42 in the browser console:

```
const ctx = document.getElementById('flag-canvas').getContext('2d');
const data = ctx.getImageData(0, 42, 400, 1).data;
let passphrase = '';
for (let i = 0; i < data.length; i += 4) {
  passphrase += String.fromCharCode(data[i]);
}
console.log('Passphrase:', passphrase); // Outputs: BEACONRED
```

3. Submit **BEACONRED** into the **Strongbox Cipher Engine** box on `/pages/beacon.html` to trigger `wasm/vault.wasm`:

Fragment III: SPARROW{pixel_by_pixel_truth}



Final Phase: Master Flag SHA-256 Calculation

To seal the vault and solve the CTF, concatenate all three fragments in sequence and compute the SHA-256 hash:

```
// 1. Concatenated String:
"SPARROW{anchor_in_the_deep}SPARROW{ghost_ship_indexed}SPARROW{pixel_by_pixel_truth}"

// 2. SHA-256 Hash Digest:
df900b8e72ef044f50cb9e248b1d9df87569d12a64c483d3b74950ce43309a47

// 3. Take first 16 Hexadecimal Characters:
"df900b8e72ef044f"
```

MASTER FLAG: SPARROW{df900b8e72ef044f}

Server API Verification

Submit the Master Flag to `POST /api/verify` via cURL or the Final Vault page:

```
curl -X POST http://localhost:3000/api/verify \
-H "Content-Type: application/json" \
-d '{"flag": "SPARROW{df900b8e72ef044f}"}'
```

Server Response (HTTP 200 OK):

```
{
  "success": true,
  "message": "The X is found. Claim the haul.",
  "flag": "SPARROW{df900b8e72ef044f}"
}
```